

PDC Assignment 4 - Building Resilient Distributed Systems

Saif Ullah | BSCS23065

PART 1: ANALYSIS

Problem 1 – Synchronization ("Lost Update")

This problem happens because both users can modify the document concurrently without using any form of concurrency control. They will receive the current version of the document, apply modifications, and save them back. Because there are no version checks or locking mechanisms employed, the latter update overwrites the first one, losing the changes in the process. It is referred to as "**Lost Update**" problem.

Problem 2 – Coordination ("Dropped Webhook")

As the webhook handler does not persist the events, if it encounters the failure in receiving webhook notifications due to network connectivity problems or server downtime, it will retry indefinitely until failing to deliver a message. and, a database record is never updated with information about the user subscription state, making the systems inconsistent with each other.

Problem 3 – Fault Tolerance ("Blocking API Calls")

LLM API calls are performed synchronously without any timeout settings implemented. If an external API response times out, a request stays stuck on that API call and blocks server capacity. Over time, the server runs out of memory, which causes the queuing and timeout problem across all requests. In this way, a slow dependency impacts the whole system.

PART 2: DESIGN

Fix 1 - Optimistic Locking (Synchronization Problem)

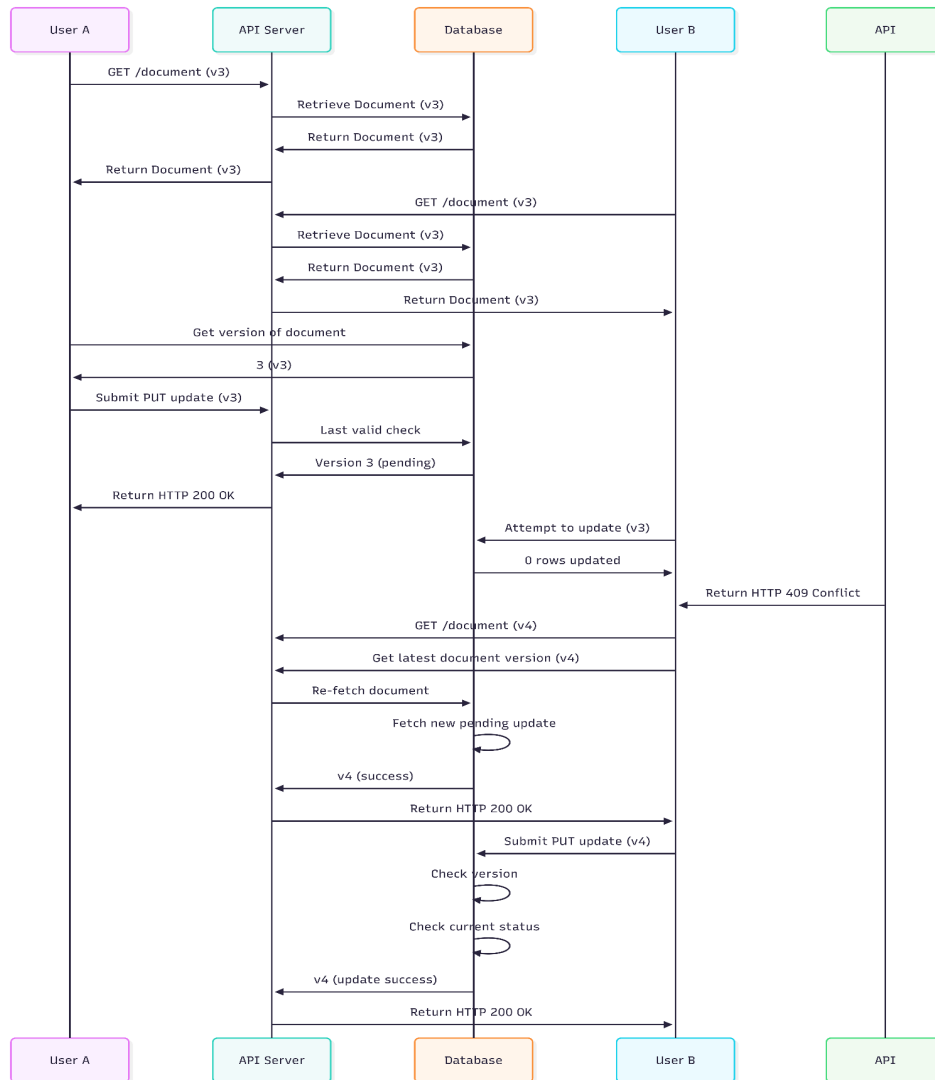
A reliable solution is to implement **optimistic locking** using version number inside the document's table.

Each document stores a `version` field. Whenever a client reads a document, the API also returns the current version number. During updates, the client must send both the modified content and the version it originally fetched. The SQL update query becomes:

```
UPDATE documents
SET content = :content,
    version = version + 1
WHERE id = :id
AND version = :expected_version;
```

If another user already updated the document, the version number will no longer match. In that case, the query updates zero rows, allowing the API to detect the conflict and return an HTTP 409 Conflict response. The client can then re-fetch the latest version, merge changes if needed, and retry safely. This prevents silent overwriting of data.

UML Sequence Diagram - Two Concurrent Users



Trade-off

This design follows a **CP (Consistency + Partition Tolerance)** approach under the CAP theorem.

The system sacrifices some availability because conflicting writes may be rejected temporarily. However, consistency is guaranteed since no update is silently lost. Users may need to retry operations, but the stored data always remains correct.

Fix 2 - Fault-Tolerant Webhook Handler (Coordination Problem)

To make webhook handling reliable, the system should store webhook events inside a `processed_webhooks` table using the Clerk webhook ID as a unique idempotency key.

The flow becomes:

1. Check whether the webhook ID already exists.
2. If it exists, immediately return HTTP 200 to avoid duplicate processing.
3. If it does not exist:
 - Apply the database state change.
 - Store the webhook ID atomically.

This makes the handler idempotent and safe against retries.

Clerk's retry mechanism can then handle temporary network failures automatically. In cases where retries are exhausted completely, failed events should be moved into a **Dead Letter Queue (DLQ)** for later inspection or recovery.

Additionally, a scheduled reconciliation job can periodically compare Clerk subscription records with the local database to repair inconsistencies.

Trade-off

This solution also follows a **CP** model. The design prioritizes consistency of billing and subscription data over immediate availability. Some operations may experience slight delays during retries or recovery, but subscription state remains reliable and correct.

Fix 3 - Circuit Breaker with Fallback (Fault Tolerance Problem)

The best solution for the blocking LLM issue is implementing a **Circuit Breaker** pattern around all external LLM API calls. The circuit breaker works in three states:

CLOSED State

Normal operation mode. All requests pass through to the LLM service. Each timeout or failure increments a failure counter. Once failures exceed a threshold (for example, 3 consecutive failures), the circuit transitions to the OPEN state.

OPEN State

The circuit immediately blocks all outgoing LLM requests. Instead of waiting for the external API, the application instantly returns a cached or fallback response. This keeps the server responsive and prevents thread exhaustion.

After a configured timeout period (for example, 10 seconds), the system moves into `HALF_OPEN` mode.

HALF_OPEN State

A small number of test requests are allowed through.

- If the request succeeds, the circuit resets back to CLOSED.
- If it fails again, the circuit returns to OPEN.

This allows safe recovery without overwhelming the external service.

Fallback Response

During outages, users can receive a simple fallback message such as: “The AI assistant is temporarily unavailable. Please try again shortly.” Although the response is not live AI output, the application remains usable and responsive.

Trade-off

This design follows an **AP (Availability + Partition Tolerance)** approach.

The system sacrifices strict consistency because users may receive cached or fallback responses instead of real-time LLM output. However, the overall application remains highly available and responsive even during external API failures.

Latency also improves significantly because failed requests are rejected immediately instead of hanging for long periods.

CAP Theorem Summary

Problem	Approach	Priority
Synchronization (Optimistic Locking)	CP	Consistency over Availability
Coordination (Reliable Webhooks)	CP	Consistency over Availability
Fault Tolerance (Circuit Breaker)	AP	Availability over Consistency

The selected approaches reflect the business importance of each component.

For document editing and subscription billing, consistency is critical because data loss or incorrect billing states are unacceptable. Therefore, the system prioritizes correctness even if some requests must be retried.

For the AI assistant feature, maintaining application availability is more important than always providing perfectly fresh responses. A degraded but responsive system is preferable to a completely unavailable application.